

Mocha Testing in Depth & Chai

Duration: 60 Minutes

Today's Journey

Mocha Basics → Async Testing → Chai Assertions → Hands-on Testing

Learning Objectives

Write structured Mocha test cases

Understand describe() and it()

Test asynchronous Node.js code

Understand done() and Promises

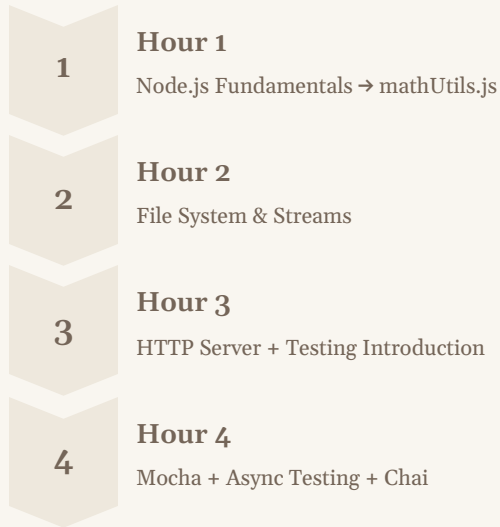
Use Chai for readable assertions

Identify failing tests and interpret test output

📌 **Trainer Note:** Tell learners: *"Today we move from simply running tests to writing tests that look like real backend application tests."*

Where We Are in the Day

Connecting the Previous Hours



Our Existing Module

```
function add(a, b) {  
  return a + b;  
}  
  
function square(n) {  
  return n * n;  
}  
  
module.exports = { add, square };
```

Today's Goal

Turn this module into a **properly tested Node.js component**.

Understanding a Mocha Test

Mocha Gives Structure to Tests

```
describe('mathUtils', () => {  
  it('adds two positive numbers', () => {  
    assert.strictEqual(add(2, 3), 5);  
  });  
});
```

Two Important Mocha Functions

describe()

- Groups related tests
- Describes the functionality being tested

it()

- Represents one test case
- Describes expected behavior

Mental Model

```
describe()  
|  
├── it() → Test Case 1  
├── it() → Test Case 2  
└── it() → Test Case 3
```

Key Idea: A test should describe **one expected behavior**.

Building the mathUtils Test Suite

Test File

test/mathUtils.test.js

```
const assert = require('assert');
const { add, square } = require('../mathUtils');

describe('mathUtils', () => {
  describe('add()', () => {
    it('adds two positive numbers', () => {
      assert.strictEqual(add(2, 3), 5);
    });

    it('handles negative numbers', () => {
      assert.strictEqual(add(-2, -3), -5);
    });
  });

  describe('square()', () => {
    it('squares a positive number', () => {
      assert.strictEqual(square(5), 25);
    });
  });
});
```

Test Organization

```
mathUtils
├── add()
│   ├── positive numbers
│   └── negative numbers
└── square()
    └── positive numbers
```

Run the Test: Green vs Red

Execute

```
npx mocha
```

Successful Test

```
mathUtils
  add()
    ✓ adds two positive numbers
    ✓ handles negative numbers
  square()
    ✓ squares a positive number

3 passing
```

Teaching Moment

Green = expected behavior

Now Break the Application!

Change:

```
function add(a, b) {
  return a - b;
}
```

Run:

```
npx mocha
```

Expected Result

```
1 failing
```

Red = defect detected

A failing test is not a bad thing. It tells us that our code does not behave as expected.

Why Async Testing Is Different

Node.js Is Heavily Asynchronous

Real applications perform:



File operations



HTTP requests



Message processing



Database queries



API calls



Timers

Example

```
function fetchUser(id, callback) {  
  setTimeout(() => {  
    callback(null, {  
      id,  
      name: 'Aditi'  
    });  
  }, 100);  
}
```

The Problem

Mocha needs to know: **"When is this asynchronous test finished?"**

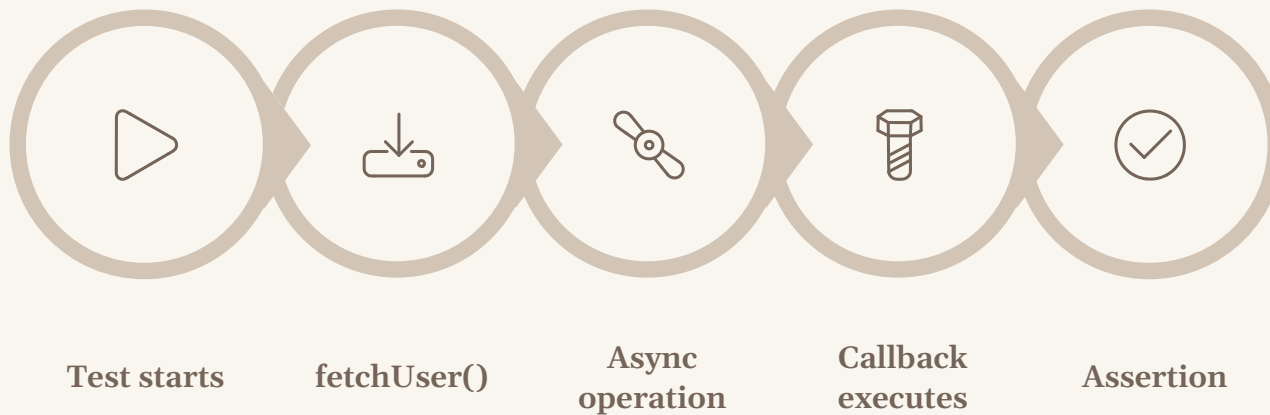
Otherwise Mocha may finish the test before the operation completes.

Async Testing with done()

Callback-Based Testing

```
it('fetches a user via callback', (done) => {  
  fetchUser(1, (err, user) => {  
    assert.strictEqual(user.name, 'Aditi');  
    done();  
  });  
});
```

How It Works



This flow shows how Mocha tracks the lifecycle of a callback-based asynchronous test from start to completion.

Important

```
done();
```

tells Mocha: "The asynchronous test has finished."

The Common done() Mistake

What Happens If We Forget done()?

```
it('fetches a user', (done) => {  
  fetchUser(1, (err, user) => {  
    assert.strictEqual(user.name, 'Aditi');  
    // done() forgotten!  
  });  
});
```

Mocha waits for completion.

Eventually:

```
Error: Timeout of 2000ms exceeded
```

Default Mocha Timeout

2000 ms

Can be changed:

```
npx mocha --timeout 5000
```

Or inside a test:

```
this.timeout(5000);
```

 **Remember:** Callback test → done()

Modern Async Testing: Promises + async/await

Promise-Based Function

```
function fetchUserAsync(id) {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve({  
        id,  
        name: 'Aditi'  
      });  
    }, 100);  
  });  
}
```

Test with async/await

```
it('fetches a user via Promise', async () => {  
  const user = await fetchUserAsync(1);  
  assert.strictEqual(user.name, 'Aditi');  
});
```

Why This Is Preferred



Mocha automatically waits for the Promise returned by an async test.

✓ **Best Practice:** Modern Node.js → Prefer async/await

Introducing Chai

What Is Chai?

Mocha answers:

"How do I run and organize tests?"

Chai answers:

"How do I express what I expect?"

Install

```
npm install --save-dev chai
```

Import

```
const { expect } = require('chai');
```

Example

Node Assert:

```
assert.strictEqual(add(2, 3), 5);
```

Chai:

```
expect(add(2, 3)).to.equal(5);
```

Chai assertions can read more naturally.

Chai expect() Assertions

Common Assertions

Equality

```
expect(add(2, 3)).to.equal(5);
```

Type

```
expect(square(4)).to.be.a('number');
```

Array

```
expect(students)  
  .to.be.an('array')  
  .with.lengthOf(2);
```

Object Property

```
expect(user)  
  .to.have.property('name')  
  .that.equals('Aditi');
```

Other Useful Assertions

```
expect(value).to.include('Node');  
expect(score).to.be.above(50);  
expect(user).to.have.property('id');  
expect(result).to.be.true;
```

equal() vs eql()

An Important Chai Concept

Consider:

```
const user1 = {  
  id: 1,  
  name: 'Aditi'  
};  
  
const user2 = {  
  id: 1,  
  name: 'Aditi'  
};
```

equal()

Checks strict identity:

```
expect(user1).to.equal(user2);
```

✗ Fails because these are different objects.

eql()

Checks deep equality:

```
expect(user1).to.eql(user2);
```

✓ Passes because their contents are equal.

Remember

equal

same value/reference

eql

same contents

Mocha + Chai: Putting It Together

Complete Test

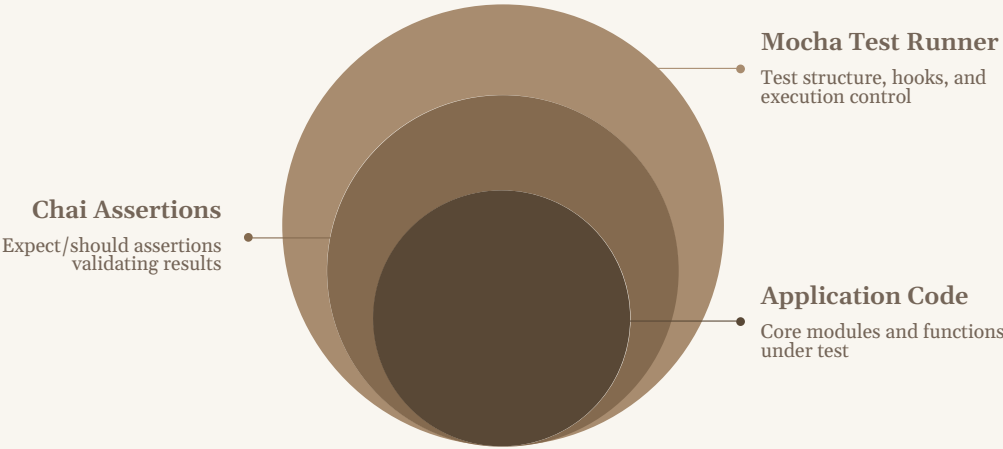
```
const { expect } = require('chai');
const { add, square } = require('../mathUtils');

describe('mathUtils with Chai', () => {
  it('adds numbers correctly', () => {
    expect(add(2, 3)).to.equal(5);
  });

  it('returns a number', () => {
    expect(square(4)).to.be.a('number');
  });

  it('calculates square correctly', () => {
    expect(square(5)).to.equal(25);
  });
});
```

Architecture



Mocha and Chai work together — Mocha provides the test runner and structure, while Chai provides expressive assertions over the application code.

Comparison

Tool	Responsibility
Mocha	Test runner & test structure
Chai	Assertions
Node assert	Built-in assertions